Gardener F5 Application Load Balancer Extension

Part 7

Package Design, Interfaces, Testing, and Implementation Roadmap

Implementation-grade low-level design for repository structure, Go contracts, test architecture, migration, delivery sequencing, and acceptance criteria.

Architectural motive: Preserve the AWS Load Balancer Controller-inspired separation established in Parts 1-6: thin Kubernetes controllers, deterministic desired-state builders, deployment managers, typed CMP clients, explicit ownership, and safe convergence.

Document status: Implementation baseline
Audience: Extension developers, reviewers, test engineers, platform operators
Scope: Service and Ingress reconciliation paths

7.0 Document Purpose and Scope

Part 7 converts the architecture and runtime behavior defined in Parts 1-6 into an implementable Go repository and delivery plan. It defines package boundaries, contracts, test seams, release gates, migration controls, and measurable acceptance criteria.

7.0.1 What this part decides

- Where each responsibility lives in the repository.
- Which Go interfaces form stable seams between Kubernetes logic and CMP logic.
- How desired state, observed state, deployment results, ownership, and errors are represented.
- How unit, contract, integration, and end-to-end tests divide responsibility.
- How the new architecture is implemented incrementally without a high-risk rewrite.
- What must be demonstrably true before each implementation milestone is accepted.

7.0.2 What this part does not redefine

- The CMP/F5 resource graph already defined in Parts 2-5.
- Service and Ingress semantic mapping already defined in Parts 3 and 4.
- Runtime state machines and recovery behavior already defined in Part 6.
- Provider features that CMP does not expose.

Figure 7-8. Requirement-to-runtime traceability

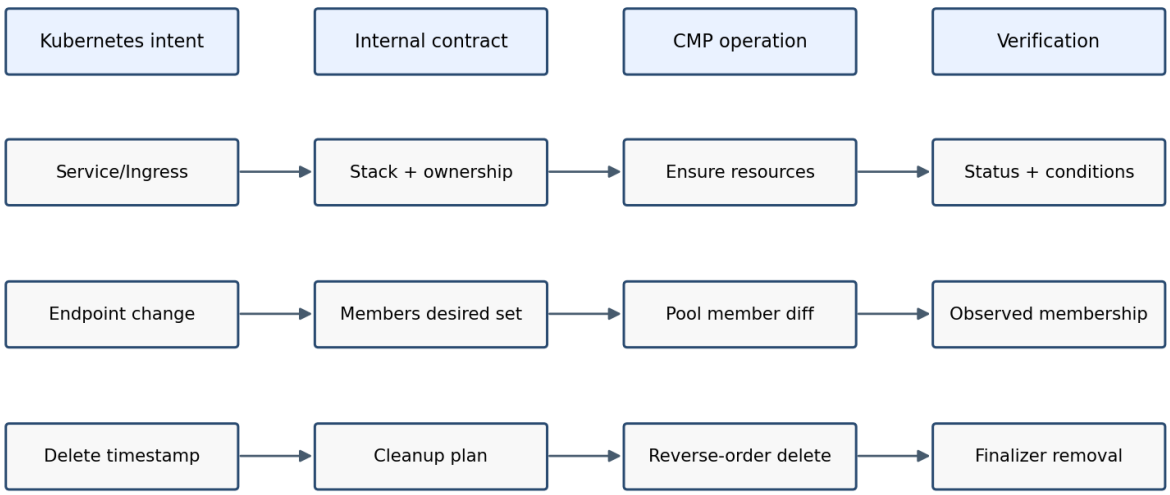


Figure 7-8. Traceability from Kubernetes intent to runtime verification.

7.1 Package Design Principles

The package design must make architectural violations difficult. The design therefore favors explicit interfaces, one-way dependencies, immutable desired-state objects, typed API boundaries, and small packages with narrow ownership.

Principle	Design consequence
Dependency direction	Higher-level orchestration depends on interfaces; lower-level CMP packages implement those interfaces. The CMP transport package never imports controller or Kubernetes API packages.
Thin reconcilers	Reconcilers fetch objects, invoke validation/build/deploy/status collaborators, and convert results into reconcile outcomes. They do not construct CMP payloads.
Pure model builders	Builders transform Kubernetes and resolved platform inputs into deterministic desired-state graphs and perform no provider writes.
Resource managers	Each CMP resource family owns discovery, comparison, ensure, readiness, and deletion logic for that family.
Typed boundaries	Raw maps, unstructured JSON, endpoint strings, and HTTP status handling remain inside typed CMP client packages.
Ownership first	Every mutation and deletion is guarded by ownership validation. Adoption is an explicit policy, never an accidental side effect.
Testable time and retries	Pollers, clocks, jitter, and backoff are injected or configurable so asynchronous behavior is deterministic in tests.

Figure 7-1. Target package layering and dependency direction

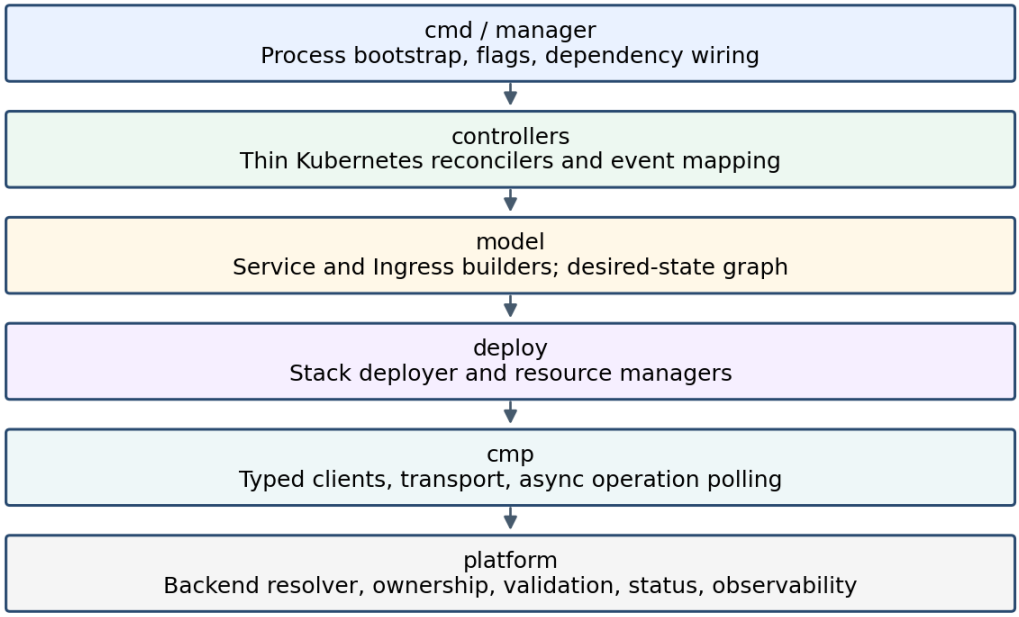
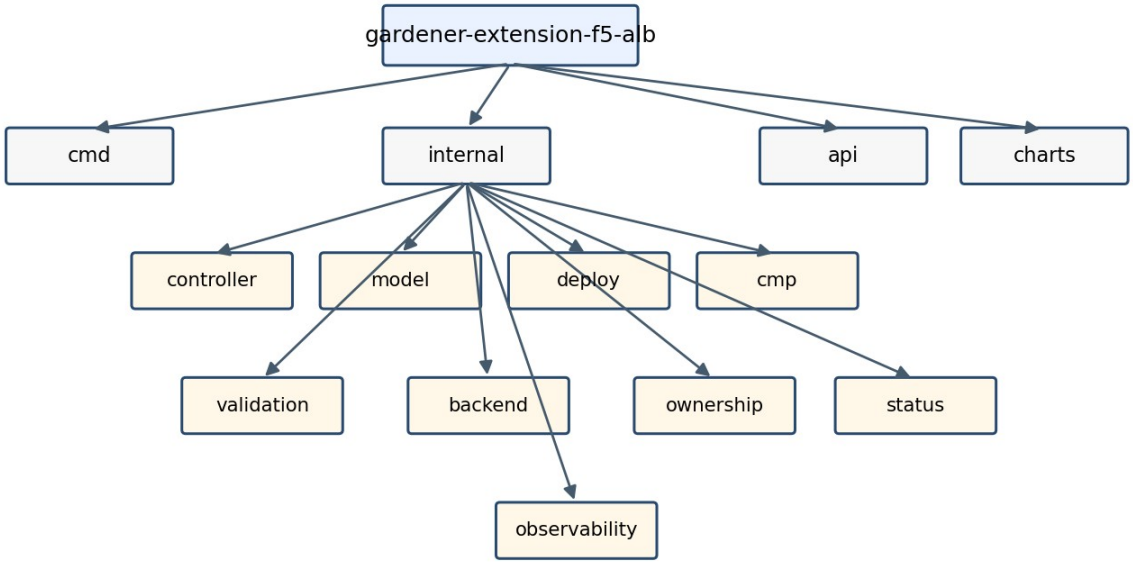


Figure 7-1. Target package layering and allowed dependency direction.

7.2 Proposed Repository Structure

The following structure is intentionally organized by architectural role rather than by individual Kubernetes resource alone. Service and Ingress share the desired-state and deployment layers while retaining separate controllers and builders.

Figure 7-2. Proposed repository structure



Dependency rule: controllers may depend on interfaces; provider transport never leaks upward.

Figure 7-2. Proposed repository structure.

```
gardener-extension-f5-alb/
├── cmd/
│   ├── controller-manager/
│   └── webhook-server/
├── api/
│   ├── config/v1alpha1/
│   └── annotations/
├── internal/
│   ├── controller/{service,ingress,common}/
│   ├── model/{core,service,ingress}/
│   ├── deploy/{stack,lbservice,vip,virtualserver,rule,pool,member,monitor,certificate}/
│   ├── cmp/{client,transport,auth,operations,models,fake}/
│   ├── backend/{resolver,node,vm,networkport}/
│   ├── ownership/
│   ├── validation/{service,ingress,common}/
│   ├── status/{service,ingress,conditions}/
│   ├── config/
│   └── observability/{metrics,events,logging,tracing}/
├── test/{fixtures,contract,integration,e2e}/
├── charts/gardener-extension-f5-alb/
├── examples/
└── docs/
```

7.2.1 Package responsibility matrix

Package	Owns	Must not own
cmd/controller-manager	Bootstrap manager, schemes, controllers, health probes, leader election, dependencies.	No reconciliation policy or CMP payload construction.
internal/controller	Kubernetes watches, request mapping, orchestration, finalizer entry/exit.	No direct HTTP or provider model mapping.
internal/model	Desired-state graph, logical IDs, builders, normalization.	No CMP calls and no status writes.
internal/deploy	Observed-state discovery, diff, dependency ordering, ensure/delete/verify.	No Kubernetes watch logic.
internal/cmp	Typed API models, auth, transport, pagination, async operations, error translation.	No Kubernetes resource semantics.
internal/backend	Endpoint, Node, VM, and network-port resolution.	No frontend resource deployment.
internal/ownership	Owner identity, tags/metadata, adoption and deletion authorization.	No resource-specific mutation.
internal/validation	Basic, semantic, and capability validation.	No provider mutation.
internal/status	Conditions, Service/Ingress status projection, event messages.	No desired-state construction.

7.3 Core Domain Types

The internal domain model is the stable contract between model builders and deployment. It must remain independent of generated CMP request/response models so provider API changes do not ripple into Kubernetes-facing code.

7.3.1 Identity and ownership

```
type OwnerRef struct {
    ClusterID  string
    Namespace  string
    Kind       string
    Name       string
    UID        types.UID
    Controller string
}

type LogicalID string

type Metadata struct {
    LogicalID LogicalID
    Name      string
    Owner     OwnerRef
    Labels    map[string]string
    ManagedBy string
}
```

LogicalID is deterministic and stable across retries. Provider IDs are observed identifiers and are never used as the primary identity of desired resources.

7.3.2 Desired stack

```
type LoadBalancerStack struct {
    Owner          OwnerRef
    LBServices     []LBService
    VIPs           []VIP
    VirtualServers []VirtualServer
    RoutingRules   []RoutingRule
    Pools          []Pool
    Members        []Member
    Monitors       []Monitor
    Certificates   []Certificate
}

type BuildResult struct {
    Stack    LoadBalancerStack
    Warnings []Warning
}
```

7.3.3 Observed state and deployment result

```
type ObservedStack struct {
    Resources map[LogicalID]ObservedResource
}

type DeploymentResult struct {
    Frontends []Frontend
    Changes   ChangeSummary
    Pending   []PendingOperation
    Warnings  []Warning
}

type ChangeSummary struct {
    Created int
    Updated int
    Deleted int
    Unchanged int
}
```

Rule: Desired objects contain configuration intent; observed objects contain provider IDs, versions, operation state, health, and timestamps. Do not merge the two representations.

7.4 Core Go Interfaces

Interfaces are introduced at architectural seams, not around every struct. They should enable substitution in tests and preserve the dependency rule without creating excessive indirection.

Figure 7-3. Core Go contracts and call direction

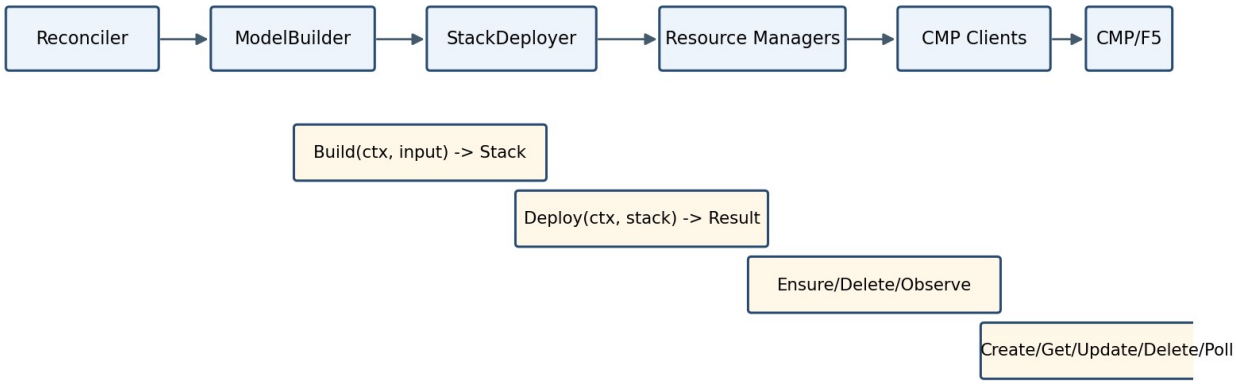


Figure 7-3. Principal Go contracts and call direction.

7.4.1 Builder contracts

```
type ServiceModelBuilder interface {
    Build(ctx context.Context, in ServiceBuildInput) (BuildResult, error)
}

type IngressModelBuilder interface {
    Build(ctx context.Context, in IngressBuildInput) (BuildResult, error)
}

type BackendResolver interface {
    ResolveService(ctx context.Context, key types.NamespacedName) ([]ResolvedBackend, error)
}
```

7.4.2 Deployment contracts

```
type StackDeployer interface {
    Deploy(ctx context.Context, desired LoadBalancerStack) (DeploymentResult, error)
    Delete(ctx context.Context, owner OwnerRef) (DeploymentResult, error)
}

type ResourceManager[TDesired any, TObserved any] interface {
    Observe(ctx context.Context, scope Scope) ([]TObserved, error)
    Ensure(ctx context.Context, desired TDesired, observed *TObserved) (EnsureResult, error)
    Delete(ctx context.Context, observed TObserved) (DeleteResult, error)
}
```

7.4.3 Ownership and validation contracts

```
type OwnershipValidator interface {
    Classify(owner OwnerRef, resource CMPResource) OwnershipDecision
}

type Validator[T any] interface {
    Validate(ctx context.Context, value T) field.ErrorList
}

type AdoptionPolicy interface {
    MayAdopt(owner OwnerRef, resource CMPResource) (bool, string)
}
```

7.4.4 Status and event contracts

```
type ServiceStatusWriter interface {
    Ready(ctx context.Context, svc *corev1.Service, result DeploymentResult) error
    Failed(ctx context.Context, svc *corev1.Service, err error) error
}

type EventPublisher interface {
    Normal(obj client.Object, reason, message string)
    Warning(obj client.Object, reason, message string)
}
```

7.5 Controller Package Design

Service and Ingress controllers share orchestration conventions but remain separate reconcilers because their eligibility, event sources, grouping semantics, status projection, and finalization differ.

7.5.1 Reconciler dependency set

```
type ServiceReconciler struct {
    Client       client.Client
    Eligibility ServiceEligibility
    Validator   Validator[ServiceValidationInput]
    Resolver    BackendResolver
    Builder     ServiceModelBuilder
    Deployer    StackDeployer
    Status      ServiceStatusWriter
    Events      EventPublisher
    Recorder    MetricsRecorder
}
```

7.5.2 Reconcile algorithm

1. Fetch the latest Kubernetes object and return successfully when it no longer exists.
2. Evaluate class/annotation eligibility before adding a finalizer.
3. For deletion, invoke stack deletion, wait for provider cleanup, then remove the finalizer.
4. Ensure the finalizer for eligible non-deleting objects.
5. Run runtime validation and publish actionable conditions/events for invalid input.
6. Resolve backends and supporting objects using read-only collaborators.
7. Build the desired stack and validate internal graph invariants.
8. Invoke the stack deployer and classify any returned error.
9. Write frontend status only after readiness requirements are met.
10. Requeue only from explicit pending operations, dependency waits, or classified retryable failures.

No hidden loops: The reconciler does not poll CMP in a blocking loop. Long operations return a bounded requeue duration and resume from observed state on the next reconciliation.

7.5.3 Watch and event mapping

Watched object	Mapping	Relevant changes
Service	Primary request	Generation, deletion timestamp, relevant annotations, loadBalancerClass, ports, traffic policy.
Ingress	Primary request/group request	Rules, TLS, class, annotations, deletion, group membership.
EndpointSlice	Map to referenced Services	Ready endpoint, nodeName, port, address changes.
Node	Map through affected resolved backends	Provider ID, addresses, readiness, deletion.
Secret	Map to referencing Ingress groups	TLS certificate/key changes.
IngressClass	Map to class users	Controller value or parameters changes.

7.6 Model Builder Package Design

Builders are deterministic transformation units. Given equivalent normalized inputs they must produce an equivalent stack regardless of event ordering, previous provider state, or controller restart.

7.6.1 Builder phases

11. Normalize Kubernetes fields and extension annotations.
12. Create the owner identity and naming context.
13. Derive LB Service and VIP intent.
14. Derive listeners/virtual servers.
15. Derive routing rules and precedence.
16. Derive pools, monitors, persistence, and certificates.
17. Attach resolved members using provider identities supplied by BackendResolver.
18. Validate graph references, uniqueness, and provider capability limits.
19. Sort collections deterministically for stable equality and test snapshots.

7.6.2 Required invariants

- Every desired resource has a unique logical ID within its owner scope.
- Every reference points to another desired resource or an explicitly external dependency.
- No CMP/provider ID is required to construct desired state.
- Ordering is stable: ports, hosts, paths, rules, pools, members, and certificates are sorted by canonical keys.
- All names satisfy provider length and character restrictions using a single naming utility.
- Unsupported semantic combinations fail before deployment.

7.6.3 Snapshot testing

Builders should be tested using compact YAML inputs and canonical JSON snapshots of the LoadBalancerStack. Snapshot updates require reviewer intent because a model change usually implies provider behavior change.

7.7 Deployment Package Design

The deployment layer owns convergence. It discovers observed resources, validates ownership, computes a plan, executes changes in dependency order, and verifies readiness without exposing HTTP details to callers.

7.7.1 Stack deployer phases

Phase	Responsibility
Discover	List or search all resources in owner scope; correlate with logical IDs and ownership metadata.
Validate	Detect collisions, foreign ownership, ambiguous duplicates, and invalid adoption candidates.
Plan	Calculate create/update/delete/no-op actions and dependency edges.
Apply	Execute creates/updates in forward dependency order; bounded concurrency only for independent resources.
Prune	Delete obsolete owned resources in reverse dependency order.
Verify	Poll or re-observe readiness; return Pending rather than blocking indefinitely.
Report	Return frontends, change summary, warnings, and pending operation tokens.

7.7.2 Manager boundaries

Manager	Owns
LBServiceManager	Top-level tenancy/container resource; owner metadata root.
VIPManager	Address reservation/configuration and frontend identity.
VirtualServerManager	Protocol/listener configuration and VIP binding.
RoutingRuleManager	Host/path conditions, priorities, and pool actions.
PoolManager	Balancing algorithm, persistence, service port semantics.
MemberManager	Backend set convergence and backend_port_id mapping.
MonitorManager	Health-monitor lifecycle and pool association.
CertificateManager	Certificate material/reference lifecycle and listener attachment.

7.7.3 Update policy

- Use normalized semantic equality, not raw JSON equality.
- Ignore provider-controlled fields and volatile timestamps.
- Prefer in-place update only for fields documented as mutable by CMP.
- For immutable changes, create replacement dependencies first, switch references, verify, then remove obsolete resources.
- Never delete an unowned resource merely because it conflicts with desired state.

7.8 Typed CMP Client Design

Typed clients form the anti-corruption layer around CMP. They isolate authentication, endpoint versions, request/response schemas, pagination, asynchronous tasks, retries, and error translation.

7.8.1 Client decomposition

```
type Clients struct {
    LBServices     LBServiceClient
    VIPs           VIPClient
    VirtualServers VirtualServerClient
    RoutingRules   RoutingRuleClient
    Pools          PoolClient
    Members        PoolMemberClient
    Monitors       MonitorClient
    Certificates   CertificateClient
    Operations     OperationClient
    Networks       NetworkClient
    Compute        ComputeClient
}
```

7.8.2 Transport responsibilities

- Ce-Auth/HMAC signing and canonical request construction.
- Base URL, tenant/project headers, correlation IDs, timeouts, TLS, and connection pooling.
- JSON encoding/decoding with bounded response bodies.
- Pagination and continuation-token handling.
- Retry only for transport-safe operations and explicitly retryable responses.
- Conversion of HTTP/CMP errors into typed domain errors.
- Redaction of credentials and certificate material from logs.

7.8.3 Client method convention

```
type PoolClient interface {
    Create(ctx context.Context, req CreatePoolRequest) (OperationRef, error)
    Get(ctx context.Context, id string) (Pool, error)
    List(ctx context.Context, filter PoolFilter) ([]Pool, error)
    Update(ctx context.Context, id string, req UpdatePoolRequest) (OperationRef, error)
    Delete(ctx context.Context, id string) (OperationRef, error)
}
```

API model rule: Generated or endpoint-specific CMP models stay in internal/cmp/models. Deployment managers map between domain desired/observed types and CMP request/response types.

7.9 Asynchronous Operations, Retry, and Error Contracts

CMP operations may be accepted before provider completion. Runtime behavior must therefore model pending work explicitly rather than treating an accepted response as readiness.

7.9.1 Operation poller

```
type OperationPoller interface {
    Observe(ctx context.Context, ref OperationRef) (OperationState, error)
}

type OperationState struct {
    Phase      OperationPhase // Pending, Running, Succeeded, Failed
    RetryAfter time.Duration
    Message    string
    ResourceID string
}
```

7.9.2 Error classification

Class	Behavior	Controller action
InvalidInput	Permanent	Write condition/event; do not retry until object changes.
Unsupported	Permanent	Explain unsupported feature and field path.
ConflictForeignOwner	Permanent/safety	Do not mutate; require operator resolution.
NotFoundDependency	Transient or pending	Re-resolve/requeue when dependency may appear.
RateLimited	Retryable	Respect Retry-After and jitter.
CMPUnavailable	Retryable	Exponential backoff with cap.
OperationPending	Pending	Requeue using operation recommendation.
OperationFailed	Classified	Retry only when failure code is retryable.
Authentication/Authorization	Configuration	Surface degraded controller condition; avoid hot loop.

7.9.3 Retry budgets

- HTTP transport retry: small bounded attempts only for idempotent requests or requests with idempotency keys.
- Reconcile retry: controller-runtime rate limiter for transient failures.
- Operation polling: provider-suggested delay or bounded progressive delay.
- No infinite synchronous polling inside a reconcile invocation.
- Emit one event per meaningful state transition, not per retry attempt.

7.10 Ownership, Adoption, and Deletion Safety

Ownership is a cross-cutting correctness boundary. Resource correlation by name alone is insufficient because names can collide, users can create similar resources, and legacy resources may exist without complete metadata.

7.10.1 Ownership identity

- Source cluster identity.
- Kubernetes source UID, namespace, kind, and name.
- Controller/extension identifier and ownership schema version.
- Logical resource ID and optional group identity for shared Ingress stacks.
- Managed-by marker distinguishable from user-owned CMP resources.

7.10.2 Decision matrix

Observed metadata	Classification	Action
Exact owner metadata	Owned	Update/delete allowed.
Same source name, different UID	Stale or conflicting	Adopt only under explicit stale-owner policy; otherwise block.
Managed-by matches, owner incomplete	Legacy candidate	Observe-only until explicit migration/adoption validation.
No ownership metadata	Foreign	Never mutate automatically.
Different controller identifier	Foreign	Never mutate/delete.
Multiple resources claim same logical ID	Ambiguous	Fail safely and require operator repair.

7.10.3 Adoption preconditions

- Adoption is enabled by configuration or a narrowly scoped annotation.
- The resource is not owned by another active Kubernetes UID/controller.
- The resource shape is compatible with the desired logical resource.
- All dependent resources can be classified consistently.
- Adoption writes ownership metadata before later destructive operations are permitted.
- An event and audit log record the adoption decision.

7.10.4 Deletion authorization

20. Re-observe the resource immediately before deletion.
21. Revalidate exact ownership and expected logical ID.
22. Check that no desired resource still references it.
23. Delete in reverse dependency order.
24. Wait for confirmed absence or terminal successful operation.
25. Remove the Kubernetes finalizer only after all controller-owned resources are absent or an explicit orphan policy is applied.

7.11 Validation Package Design

Validation is split into admission-time validation and reconcile-time validation. Admission improves user feedback, while reconcile-time validation remains authoritative because webhooks can be bypassed and runtime dependencies change.

7.11.1 Validation layers

Layer	Examples	Execution point
Basic/schema	Presence, format, enum, range, mutually exclusive annotations.	Webhook + reconcile
Kubernetes semantic	Port references, Service existence, TLS Secret shape, path type, class rules.	Webhook where possible + reconcile
CMP capability	Supported protocols, routing limits, persistence/monitor combinations, certificate constraints.	Webhook with static capabilities + reconcile
Runtime dependency	Endpoint/Node/VM/network port resolution, secret readability, provider connectivity.	Reconcile only
Desired graph	Unique IDs, references, priorities, ownership scope, naming constraints.	Builder/deploy boundary

7.11.2 Error presentation

- Return field-specific admission errors for user-editable fields.
- Use stable condition reasons such as InvalidConfiguration, UnsupportedFeature, BackendResolutionFailed, ProviderConflict, and ProviderUnavailable.
- Keep messages actionable and avoid exposing credentials or raw provider payloads.
- Aggregate independent validation failures but stop before provider mutation.

7.12 Observability and Operational Contracts

Operational behavior is part of the implementation contract. Logs, metrics, events, and health endpoints must make each reconciliation phase and provider dependency diagnosable.

7.12.1 Structured logging fields

- controller, namespace, name, UID, generation, reconcile ID
- owner/group ID, logical resource ID, CMP resource type and provider ID
- phase: validate, resolve, build, discover, plan, apply, verify, status, finalize
- operation ID and classified error type
- Never log secret bodies, private keys, auth signatures, or full token-bearing headers.

7.12.2 Metrics

Metric	Labels	Purpose
reconcile_total / duration	controller, result	Throughput and latency.
desired_resources	kind	Stack size and cardinality.
cmp_requests_total / duration	operation, resource, result	Provider dependency behavior.
cmp_operations_pending	resource	Async backlog.
deployment_changes	action, resource	Create/update/delete churn.
ownership_conflicts_total	resource	Safety/collision signal.
backend_resolution_failures_total	stage	Node/VM/network mapping health.
status_update_failures_total	kind	Kubernetes API/status reliability.

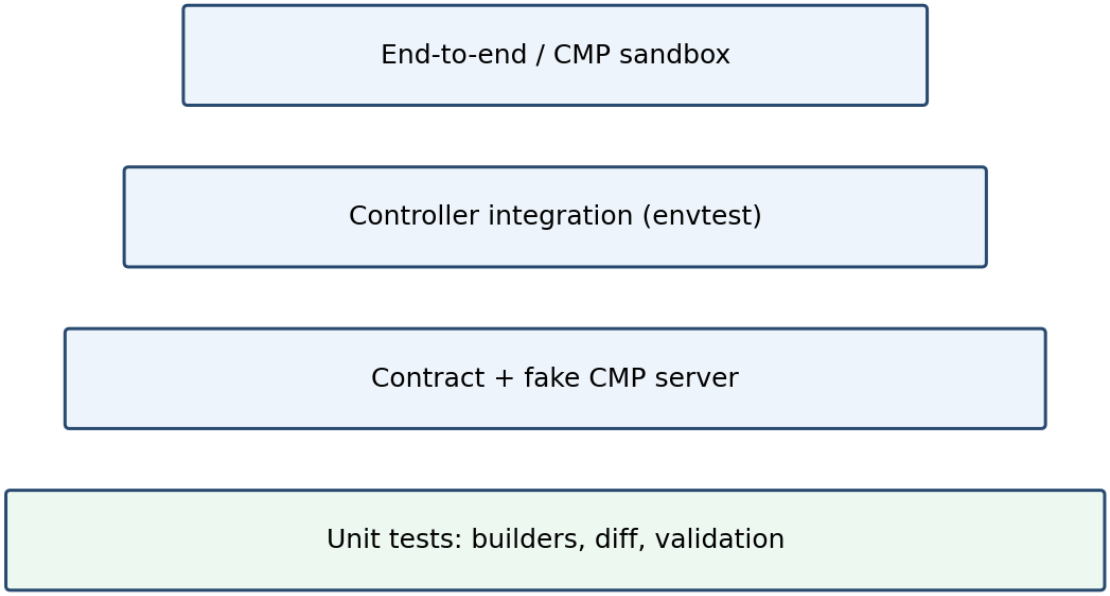
7.12.3 Health and readiness

- Liveness verifies process responsiveness only.
- Readiness verifies caches are synced and required configuration is loaded.
- CMP temporary unavailability should not necessarily kill the process; expose it through metrics/conditions and retry.
- Leader election is enabled for active controllers; webhooks remain horizontally available.

7.13 Test Strategy

The test strategy mirrors package boundaries. Most semantic behavior is validated with fast deterministic tests; fewer tests cross Kubernetes and CMP boundaries.

Figure 7-4. Test strategy pyramid



Most behavior is proven below the API boundary; only a small number of expensive E2E tests.

Figure 7-4. Test strategy pyramid.

7.13.1 Unit tests

- Model builder fixtures for single-port, multi-port, TLS, host/path, default backend, grouping, traffic policy, session affinity, and unsupported cases.
- Deterministic logical IDs, names, ordering, rule priorities, and references.
- Ownership classification and adoption/deletion decisions.
- Diff normalization, immutable-field replacement plans, and dependency ordering.
- Error classification, retry decisions, backoff, and operation state transitions.
- Status condition transition behavior and message stability.

7.13.2 CMP contract tests

- Run typed clients against an HTTP fake that validates method, path, headers, HMAC/auth construction, request schema, and response decoding.

- Exercise pagination, empty responses, malformed bodies, conflict, rate limit, timeout, async operation, and terminal failure cases.
- Keep recorded fixtures derived from the approved Swagger/OpenAPI version; fail when generated/request models drift unexpectedly.
- Never make unit tests depend on a live shared CMP environment.

7.13.3 Controller integration tests

- Use controller-runtime envtest with CRDs/webhooks installed.
- Create Kubernetes resources and assert finalizers, status, conditions, events, and calls to fake deployment collaborators.
- Validate watch mapping from EndpointSlice, Secret, Node, IngressClass, Service, and grouped Ingress changes.
- Exercise deletion, conflict, retryable error, generation change during reconcile, and controller restart/re-observation.

7.13.4 End-to-end tests

- Service creation exposes a reachable frontend and healthy backend.
- Ingress host/path and default-backend routing behaves as specified.
- TLS certificate is installed and rotated without prolonged outage.
- Scale up/down and node replacement converge membership correctly.
- Partial provider failure recovers without duplicate resources.
- Deletion removes only owned resources and clears finalizers.
- Upgrade/migration preserves existing frontends or follows the declared replacement policy.

7.13.5 Failure-injection matrix

Injected failure	Required assertion
CMP timeout during create	No duplicate logical resource; retry/re-observe.
Create accepted, response lost	Discover/adopt exact owned result by metadata/idempotency key.
Pool created, member creation fails	Next reconcile resumes from pool and converges members.
Certificate invalid	No listener cutover; clear condition and event.
Foreign VIP collision	No mutation/deletion; ownership conflict condition.
Endpoint changes during deployment	Next reconcile builds newer generation; no stale status overwrite.
Controller restart	Observed provider state reconstructs progress.
Delete operation remains pending	Finalizer retained; bounded requeue.

7.14 Test Fixtures, Fakes, and Golden Data

Test support is a product of the architecture, not an afterthought. Fakes should model contract behavior without reproducing the complete CMP implementation.

7.14.1 Fixture organization

```
test/fixtures/
├── kubernetes/
│   ├── services/
│   ├── ingresses/
│   ├── endpointslices/
│   ├── nodes/
│   └── secrets/
├── desired-stacks/
├── cmp-responses/
├── operations/
└── ownership/
```

7.14.2 Fake design

- Fake typed clients record calls and support scripted responses/errors.
- HTTP contract server validates wire behavior independently of manager behavior.
- Fake clock and jitter source control operation polling and retry tests.
- Provider IDs are deterministic in tests but never embedded in desired snapshots.
- Golden data includes an explicit schema/version so migrations are reviewable.

7.15 Continuous Integration and Quality Gates

Every pull request must prove package-level correctness and preserve API/behavior contracts. Expensive tests run after fast local gates, but no release bypasses integration and smoke verification.

Figure 7-6. Continuous integration quality gates

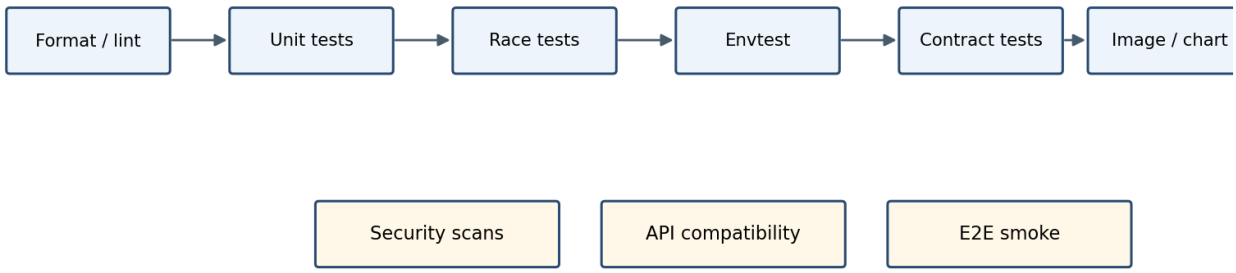


Figure 7-6. Continuous integration quality gates.

7.15.1 Mandatory pull-request gates

- gofmt/goimports, go vet, staticcheck/golangci-lint, forbidden dependency checks.
- Unit tests with race detector for concurrency-sensitive packages.
- Controller envtest integration suite.
- CMP client contract tests against the fake server.
- API/config compatibility and generated code cleanliness.
- Helm lint/template, manifest validation, container image build, vulnerability/license scanning.
- Coverage thresholds applied by critical package rather than only repository-wide percentage.

7.15.2 Release gates

- End-to-end smoke against a representative CMP/F5 environment.
- Upgrade test from previous released version.
- Rollback procedure verified.
- No unresolved critical security findings.
- Operator runbook and compatibility matrix updated.
- Known limitations match validation behavior and documentation.

7.16 Migration and Compatibility Plan

The new architecture should replace behavior incrementally. Migration must prioritize resource safety, stable external addresses, and the ability to compare old and new decisions before changing provider state.

Figure 7-7. Safe migration and cutover modes

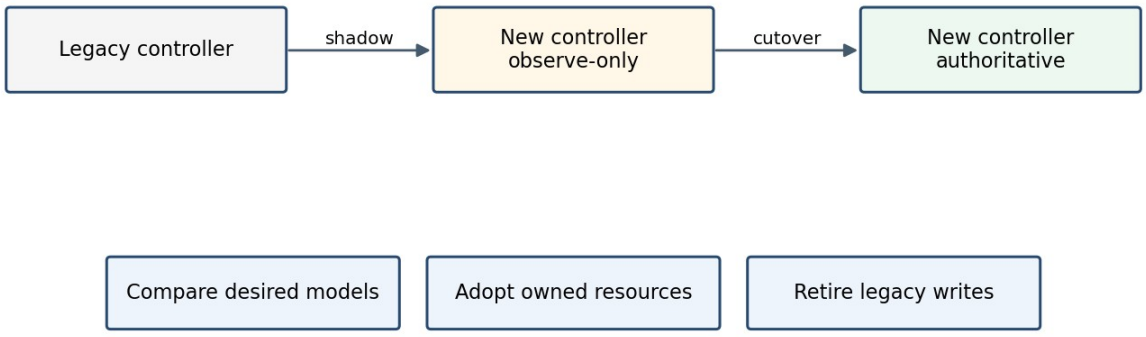


Figure 7-7. Recommended migration and cutover modes.

7.16.1 Compatibility controls

- Feature flags for Service, Ingress, adoption, grouped Ingress, and destructive pruning.
- Ownership schema version and legacy metadata reader.
- Configuration conversion/defaulting for existing installations.
- Observe-only/shadow mode that builds desired state and discovers CMP resources but performs no mutation.
- Per-object opt-in through loadBalancerClass, IngressClass, or migration annotation.
- Rollback keeps the last compatible ownership metadata readable by the previous version.

7.16.2 Migration sequence

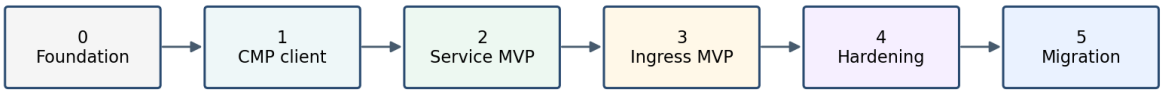
26. Inventory existing Kubernetes objects and CMP resources; classify ownership completeness.
27. Run the new builders in shadow mode and compare desired graphs with current effective state.
28. Enable read-only provider discovery and report collisions/ambiguity.
29. Adopt only resources satisfying explicit adoption preconditions.
30. Cut over a limited namespace/class cohort to authoritative reconciliation.
31. Observe error rate, resource churn, frontend stability, and cleanup safety.

32. Expand cohorts, then disable legacy writes.
33. Retain rollback support until all managed resources carry the new ownership schema.

7.17 Incremental Implementation Roadmap

The roadmap delivers vertical slices rather than building every package in isolation. Each phase produces demonstrable behavior and leaves the repository releasable behind controlled feature gates.

Figure 7-5. Incremental implementation roadmap



Each phase has an independently demonstrable acceptance gate and can be released behind feature flags.

Figure 7-5. Incremental implementation roadmap.

Phase	Deliverables	Exit gate
Phase 0 - Foundation	Repository skeleton; config API; manager bootstrap; common domain types; naming/logging/metrics; feature flags; CI baseline.	Binary starts, health/readiness pass, packages obey dependency checks, empty controllers run safely.
Phase 1 - CMP client and provider foundations	Transport, authentication, typed clients, API models, operations poller, error classifier, fake HTTP server, ownership metadata codec.	Contract tests cover every required CMP endpoint and failure class; secrets are redacted.
Phase 2 - Service MVP	Service eligibility, finalizer, validation, EndpointSlice/backend resolver, Service builder, LB/VIP/VS/pool/member/monitor managers, status.	Single- and multi-port LoadBalancer Services create, update, scale, recover, and delete safely.
Phase 3 - Ingress MVP	IngressClass, host/path/default backend, TLS/certificates, routing rules, status, basic grouping.	HTTP/HTTPS routing and certificate lifecycle pass integration/E2E tests.
Phase 4 - Hardening	Adoption, immutable replacements, partial-failure recovery, concurrency controls, rate limiting, comprehensive observability, performance tests.	Restart and injected-failure scenarios converge without duplicate or foreign deletion.
Phase 5 - Migration and release	Shadow mode, legacy metadata compatibility, cohort cutover, upgrade/rollback, runbooks and compatibility documentation.	Production pilot meets SLO and safety gates; legacy write path can be retired.

7.18 Implementation Work Breakdown

Tasks are ordered to minimize rework and expose architectural integration issues early. Each task should include code, tests, metrics/logging, and documentation where applicable.

ID	Task	Area
P0.1	Create package skeleton and dependency-lint rule.	Foundation
P0.2	Define config API, defaults, validation, feature flags.	Foundation
P0.3	Define domain IDs, owner metadata, desired/observed stack.	Foundation
P1.1	Implement CMP transport, auth, timeout, redaction.	CMP
P1.2	Implement typed resource clients and operation client.	CMP
P1.3	Implement error classifier, retry policy, fake server.	CMP
P1.4	Implement ownership codec/classifier/adoption policy.	Safety
P2.1	Implement Service controller orchestration/finalizer/status.	Service
P2.2	Implement EndpointSlice -> Node -> VM -> port resolver.	Service
P2.3	Implement Service validation and model builder.	Service
P2.4	Implement stack deployer and core resource managers.	Deployment
P2.5	Implement Service integration and E2E tests.	Testing
P3.1	Implement IngressClass/group controller and mapping.	Ingress
P3.2	Implement Ingress validation/builder/rule priorities.	Ingress
P3.3	Implement certificate and routing-rule managers.	Ingress
P3.4	Implement Ingress integration and E2E tests.	Testing
P4.1	Implement replacement planner and partial-failure resume.	Hardening
P4.2	Implement rate limits, concurrency, performance tests.	Hardening
P4.3	Complete dashboards, alerts, events, runbooks.	Operations
P5.1	Implement shadow mode and comparison reports.	Migration
P5.2	Implement legacy adoption and cohort cutover controls.	Migration
P5.3	Run pilot, upgrade, rollback, and release qualification.	Release

7.19 Acceptance Criteria

Acceptance criteria are behavioral and measurable. Completion is not defined solely by merged code or successful happy-path creation.

7.19.1 Architecture and package criteria

- Controllers contain no direct CMP HTTP calls or provider payload construction.
- Model packages import neither CMP client packages nor controller packages.
- CMP transport and endpoint models are isolated below typed clients.
- Every mutating resource manager enforces ownership before update/delete.
- All long-running provider operations return pending/requeue state rather than blocking indefinitely.
- Desired-state output is deterministic for equivalent inputs.

7.19.2 Functional criteria

- Service: create, multi-port update, backend scale, node replacement, status, and deletion pass.
- Ingress: class selection, host/path routing, default backend, TLS/certificate rotation, grouping, status, and deletion pass.
- Unsupported cases are rejected before mutation with field-specific reasons.
- Controller restart at any deployment phase resumes safely from observed state.
- Foreign or ambiguous resources are never modified or deleted.

7.19.3 Reliability and performance criteria

- No duplicate provider resources after lost responses, retries, or controller restart in failure-injection tests.
- Provider unavailability does not cause unbounded hot loops or process failure.
- Reconciliation latency and CMP request count remain within agreed environment-specific SLOs.
- Independent resources may be reconciled concurrently within configured global and tenant limits.
- Deletion finalizers remain until confirmed cleanup and do not disappear on transient CMP failure.

7.19.4 Test and release criteria

- Critical builder, ownership, diff, and error-classification packages meet agreed branch coverage thresholds.
- Race detector passes for controllers, deployer, operation polling, and fake clients.
- All required CMP endpoint contracts are covered by wire-level tests.
- Envtest suite validates watches, finalizers, conditions, and status conflict retries.
- E2E smoke, upgrade, and rollback tests pass against the qualified CMP/F5 environment.
- Runbooks document ownership conflict, stuck operation, failed deletion, certificate error, and backend-resolution incidents.

7.20 Definition of Done by Component

This checklist provides a review gate for each implementation area and prevents partially integrated components from being treated as complete.

Component	Definition of done
Controller	Watches mapped; finalizer; classified retries; status/events; envtest; no CMP imports.
Builder	Deterministic; graph invariants; supported/unsupported fixtures; snapshot tests.
Resource manager	Observe/diff/ensure/delete/verify; ownership checks; async handling; unit tests.
Typed client	Typed requests/responses; auth; pagination; timeout; error mapping; contract tests.
Backend resolver	Endpoint readiness; Node/VM/port mapping; cache strategy; error classification; tests.
Validation	Admission and reconcile coverage; field paths; stable reasons; capability tests.
Status writer	Observed generation; transition-aware conditions; conflict retry; no stale overwrite.
Feature	Docs, metrics, logs, event reasons, runbook, upgrade/rollback consideration.

7.21 Key Risks and Mitigations

The roadmap deliberately front-loads provider contract and ownership work because these are the highest-risk foundations for all subsequent resource behavior.

Risk	Mitigation
CMP API semantics differ from Swagger or vary by environment	Contract tests against qualified environments; capability discovery/version matrix; isolate differences in typed clients.
Ownership metadata cannot be stored on every resource	Use supported descriptions/tags plus parent correlation; maintain strict adoption policy; never rely on name alone for deletion.
Async operations lack reliable idempotency	Re-observe before retry; deterministic names/logical IDs; idempotency keys where supported; ambiguous-state error handling.
Ingress grouping causes cross-object blast radius	Stable group ownership, deterministic aggregate build, group-level status/error policy, controlled rollout.
Backend VM/network mapping is slow or inconsistent	Resolver cache with invalidation, batched queries, explicit stale behavior, metrics by resolution stage.
Large stacks cause API churn	Semantic diff, stable ordering, bounded concurrency, incremental member updates, request metrics/performance tests.
Migration accidentally deletes legacy resources	Observe-only phase, explicit adoption, pruning feature flag, ownership schema validation, cohort rollout.

7.22 Final Implementation Blueprint

The complete implementation remains aligned with the original motive: Kubernetes intent is transformed into a provider-independent desired model; deployment managers converge CMP/F5 resources through typed clients; ownership and validation guard every mutation; runtime state is observable and recoverable.

Figure 7-1. Target package layering and dependency direction

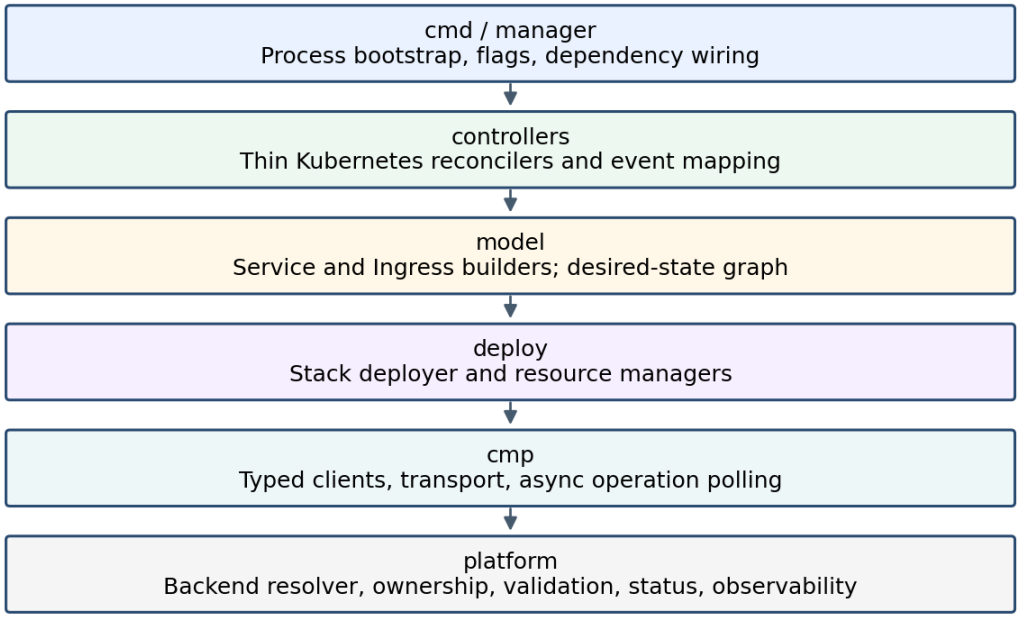


Figure 7-1 (repeated). Final package layering.

7.22.1 End-to-end contract

34. Kubernetes event mapping produces a bounded reconcile request.
35. The reconciler validates eligibility and lifecycle state.
36. Read-only collaborators resolve dependencies and backends.
37. A pure builder creates a deterministic LoadBalancerStack.
38. The stack deployer discovers observed CMP state and validates ownership.
39. Resource managers converge the graph using typed CMP clients.
40. Pending asynchronous operations result in controlled requeue.
41. Verified frontends and conditions are projected back to Kubernetes status.
42. Deletion follows reverse dependencies and removes finalizers only after safe cleanup.

7.22.2 Implementation checklist

- Repository/package skeleton and dependency rules
- Domain desired/observed contracts and logical IDs
- Typed CMP clients and operation/error handling

- Ownership/adoption/deletion safety
- Service vertical slice
- Ingress vertical slice
- Unit, contract, envtest, and E2E suites
- Observability, CI gates, migration controls, runbooks
- Acceptance evidence and release qualification

Part 7 conclusion: This package and delivery design is not a separate architecture. It is the executable form of Parts 1-6 and preserves their central separation of concerns throughout implementation and testing.

Appendix A - Suggested Interface Review Checklist

- Does the interface represent a genuine architectural seam?
- Is the interface owned by the consumer package?
- Can it be exercised without a live CMP environment?
- Does it leak HTTP, JSON, Swagger-generated, or Kubernetes client details unnecessarily?
- Are context cancellation and typed errors preserved?
- Is the result sufficient for idempotent retry and status projection?
- Can future CMP version differences remain behind the implementation?

Appendix B - Suggested Pull Request Sequence

43. Foundation types and package dependency guard.
44. CMP transport/auth plus contract fake.
45. Typed clients and operation/error model.
46. Ownership and naming utilities.
47. Stack deployer framework with one minimal manager.
48. Service builder/controller vertical slice.
49. Remaining Service resource managers and resilience tests.
50. Ingress builder/controller vertical slice.
51. Routing/certificate/grouping and resilience tests.
52. Shadow migration, operational dashboards, release gates.